randomized treap, and check, after performing the same sequence of operations on two instances of those two containers, how the heights of the two trees compares.

This measure will give us the gist of the asymptotic improvement of balanced trees over BSTs, because, as we discussed, operations on binary trees (balanced or not) require a number of steps proportional to the height of the tree.

We also know, however, that asymptotic analysis discards the constant coefficients, hiding out the code complexity that usually comes with more advanced algorithms: therefore, having an indication of the actual running time will provide more thorough information that can help us choose the best implementation for our applications.

In the tests, that you can find on the books GitHub repo, we try three different scenarios:

- We create large trees of increasing (random) size, whose keys are random integers, with an initial sequence of insertions, followed by random removal and insertion (with a rate of 1:1).
- We proceed similarly to the previous step, but the possible key values are limited to a small subset (f.i., just 0…100, forcing several duplicates in the tree).
- We insert an ordered sequence of numbers into the trees.

The results of the first test we ran are shown in figure 3.17: you can see that the height of both trees is growing logarithmically… at first this seems rather discouraging, as there seems to be no improvement given by using random treaps over plain BSTs.

We need, however, to make a couple of considerations.